



Problem Solving With C
UE24CS151B

By,
Prof. Bivas Bhattacharya,
Assistant Professor

Question 1:

- **Question:** Explain the rules for creating valid identifiers in C. Why are these rules important? Give valid and invalid examples and explain why the invalid examples are not allowed.
- **Answer:**
- Identifiers are names given to variables, functions, keywords, and other user-defined elements in C. The rules ensure the compiler can correctly interpret and process the code.
- **Rules for Valid Identifiers:**
- Must start with a letter (A-Z, a-z) or an underscore (_).
- Can be followed by zero or more letters, underscores, or digits (0-9).
- Cannot contain punctuation characters such as #, @, and %.
- **Examples:**
- **Valid:** myVariable, _count, student_ID, data123
- **Invalid:** 123data (starts with a digit), my-Variable (contains a hyphen), int (is a keyword), data# (contains a #).
- **Why Invalid Examples Are Not Allowed:** The invalid examples violate the syntax rules of the C language. Starting with a digit would cause the compiler to misinterpret it as a number, containing hyphens makes it difficult to separate the identifiers, keywords are reserved words for C, and special characters cannot be present in an identifier. Using them would lead to compilation errors because the compiler would not be able to recognize the construct as an identifier.

Question 2:

- **Question:** Explain the difference between variable declaration, definition, and initialization in C. Provide code examples to illustrate each concept.
- **Answer:**

- Declaration, definition, and initialization are distinct but related concepts in C programming related to variables.
- **Declaration:** A declaration informs the compiler about the existence of a variable, specifying its name and data type. It does not allocate memory for the variable.
- **Example:** `int myNumber;`
- **Definition:** A definition allocates memory space for the variable. In many cases, a declaration also serves as a definition.
- **Example:** `int myNumber;` (This declaration also serves as the definition because it allocates memory for the integer variable.)
- **Initialization:** Initialization assigns an initial value to the variable when it is declared or defined. This ensures that the variable starts with a known value.
- **Example:** `int myNumber = 10;` (This declares, defines, and initializes the variable `myNumber` with the value 10.)

Question 3:

- **Question:** What are keywords in C? Why can't they be used as variable names? Provide five examples of C keywords and explain what each one is used for.
- **Answer:**
- Keywords are reserved identifiers in C that have a predefined meaning to the compiler. They form the core vocabulary of the language.
- Keywords cannot be used as variable names because the compiler would misinterpret them. If a keyword were used as a variable name, the compiler would not be able to distinguish between the intended variable and the keyword's predefined meaning, leading to syntax errors.
- **Examples:**
- `int`: Specifies an integer data type. `int age = 25;`
- `float`: Specifies a floating-point data type. `float pi = 3.14159;`
- `for`: Used to create a loop. `for (int i = 0; i < 10; i++) { ... }`
- `if`: Used to create a conditional statement. `if (age >= 18) { ... }`
- `return`: Used to return a value from a function. `return 0;`

Question 4:

- **Question:** Describe the primary data types in C (`int`, `float`, `double`, and `char`). Explain the purpose of each type and the kind of data they are used to store.
- **Answer:**
- The primary data types in C are fundamental building blocks for representing different kinds of data.
- `int`: Used to store integer values (whole numbers) without any fractional part. It is typically used for counting, indexing, and representing discrete quantities.

- **Example:** `int count = 10;`
- `float`: Used to store single-precision floating-point numbers, which are numbers with a fractional part. It is typically used for representing real numbers with limited precision.
- **Example:** `float price = 99.99;`
- `double`: Used to store double-precision floating-point numbers, which offer greater precision than `float`. It is preferred over `float` when higher accuracy is required.
- **Example:** `double pi = 3.14159265359;`
- `char`: Used to store single characters, such as letters, digits, or symbols. It is typically used for representing text and character-based data.
- **Example:** `char grade = 'A';`

Question 5:

- **Question:** Explain the difference between primary and secondary data types in C. Give examples of derived and user-defined secondary data types, including how each is defined.
- **Answer:**
- Primary data types are the fundamental, built-in data types provided by the C language (like `int`, `float`, `char`, `double`). Secondary data types are derived from primary data types or defined by the user. **Derived data types:** These are data types that are created by combining primary data types.
- **Example:** Arrays. An array is a collection of elements of the same data type.
- **Definition:** `int numbers;` (This declares an array named `numbers` that can hold 5 integers.)
- **User-defined data types:** These are data types that are defined by the user using keywords like `struct`, `enum`, `union`, and `typedef`.
- **Example 1:** `struct`. A structure is a collection of variables of different data types grouped together under a single name.
- **Definition:**

```
C
struct Student {
    char name[50];
    int age;
    float gpa;
};
```

- **Example 2:** `enum`. Enumerated types are user-defined types where all possible values are defined as named constants.
- **Definition:**

```
C
enum Day {
    SUNDAY,
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
```

SATURDAY

};